ICT 1306
OBJECT ORIENTED PROGRAMMING

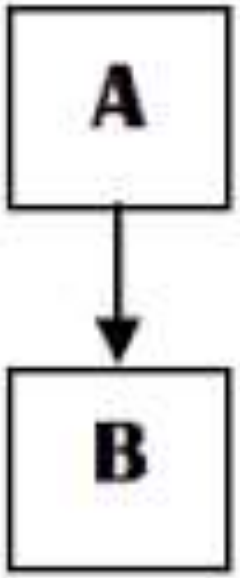
6.INHERITANCE

TC Irugalbandara
KHA Hettige

Inheritance

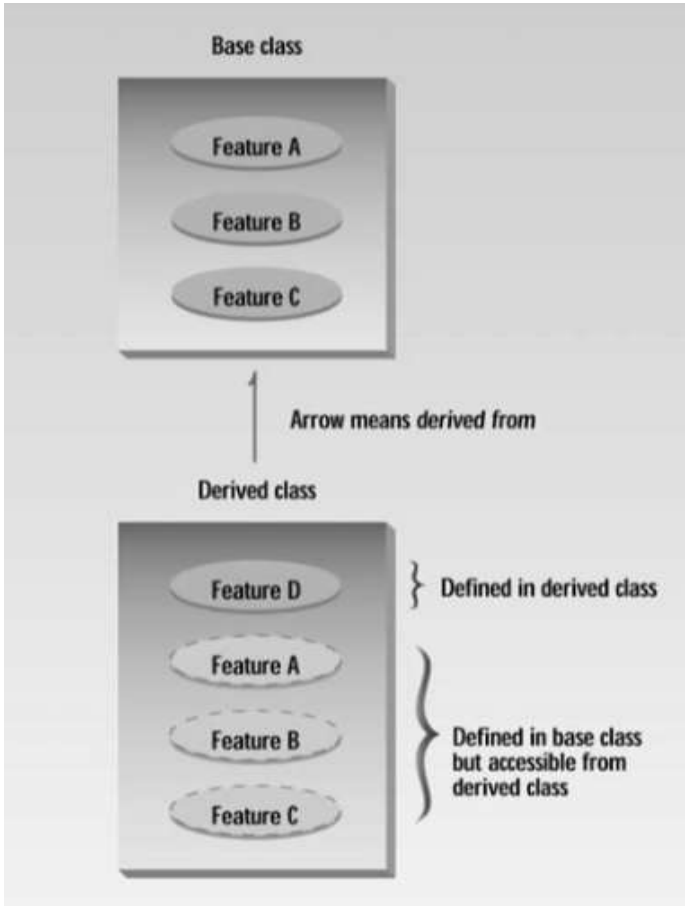
- Classes in C++ can be extended, creating new classes which retain characteristics of the base class. This process, known as inheritance.
- Inheritance allows to define a class in terms of another class, which makes it easier to create and maintain an application.
- It is the process of creating new classes, called **derived classes**, from existing **base classes**.
- The derived class inherits all the capabilities of the base class but can add its own features. The base class is unchanged by this process.
- This provides an opportunity to **reuse** the code functionality and **fast** implementation.
- The idea of inheritance implements the "**is a"** relationship.
- Ex: Dog is a Animal.
  Car is a Vehicle.

Base and Derived Relationship



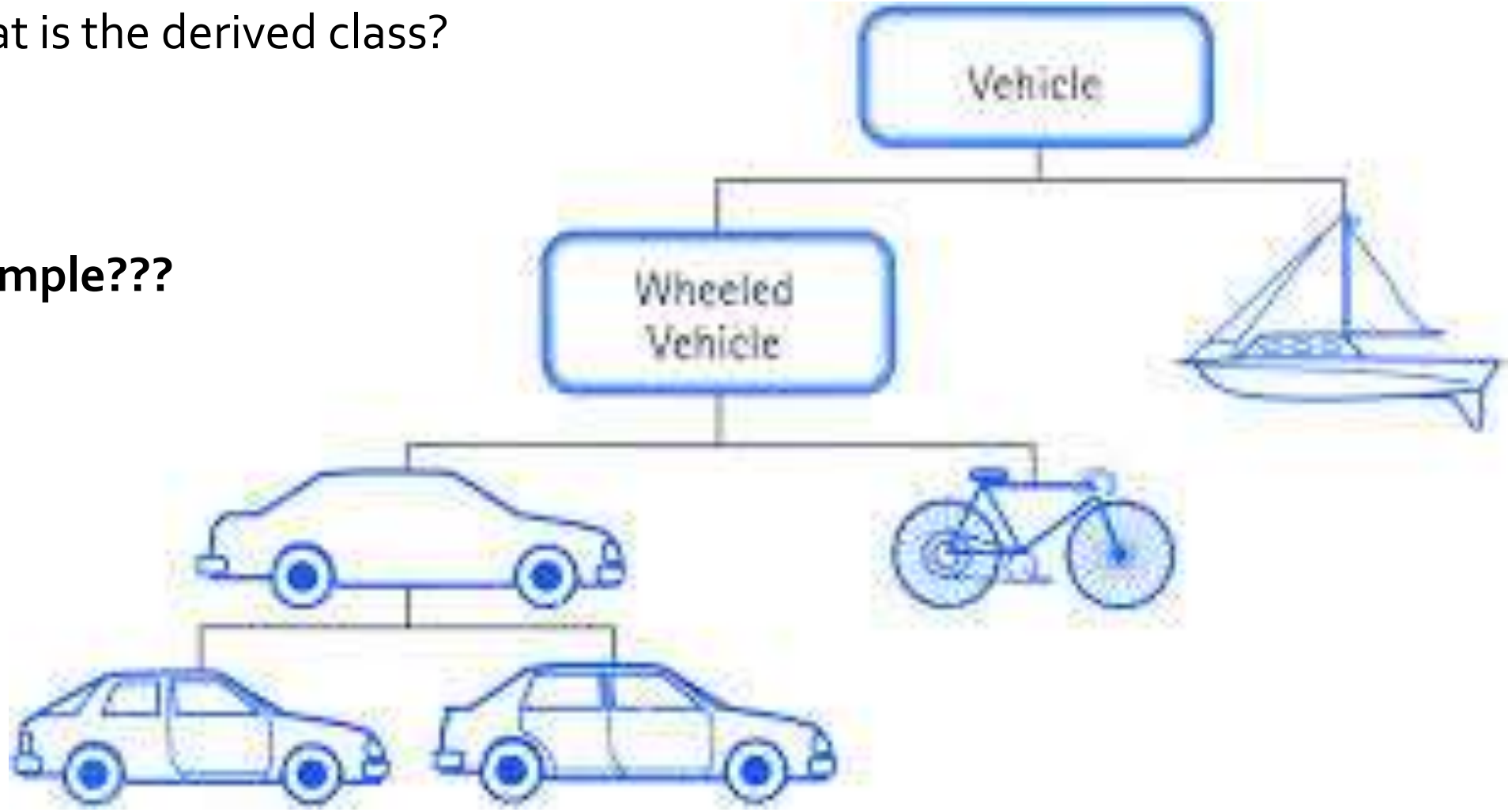
Base Class

Derived Class



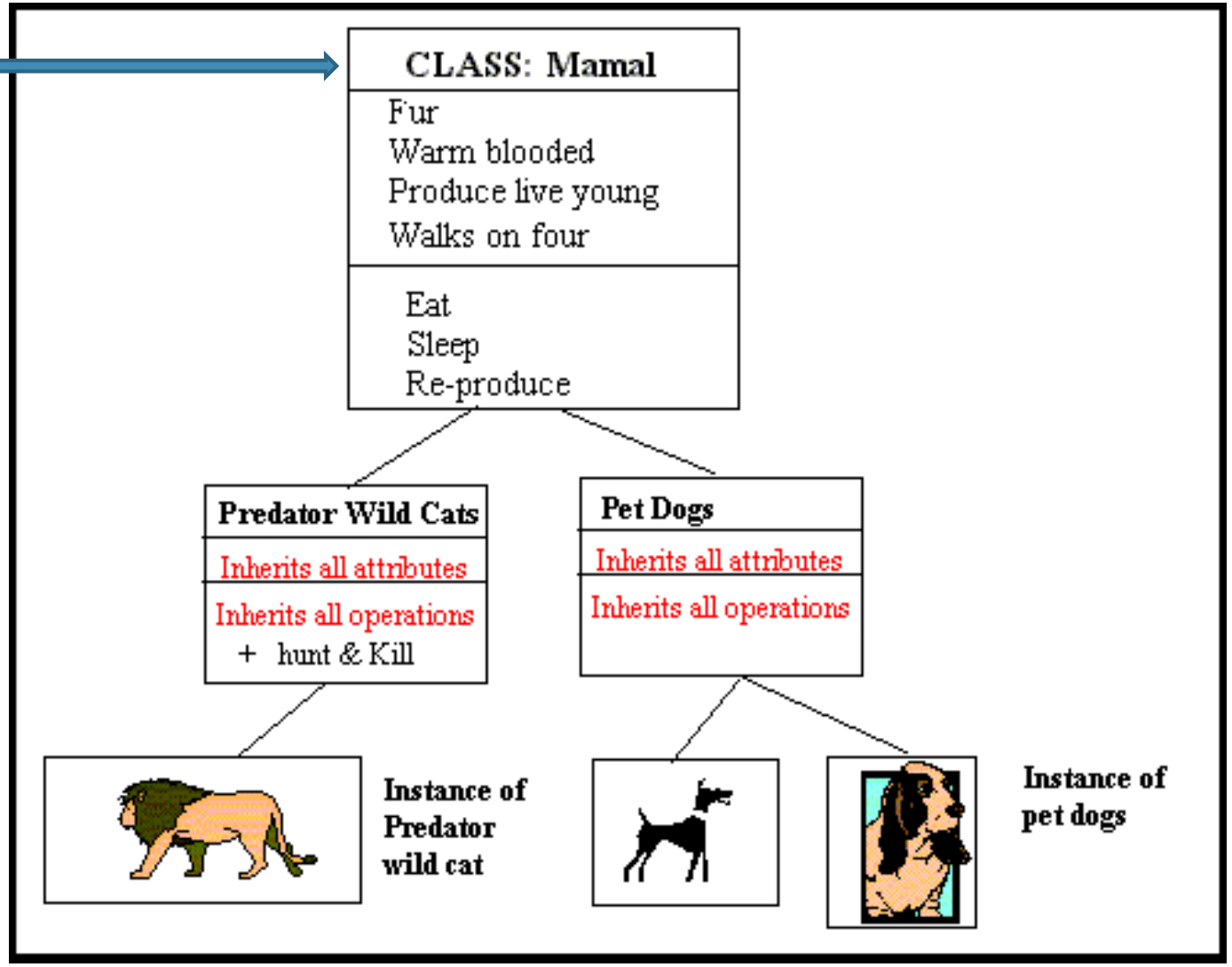
What is the base class?
What is the derived class?

Example???



Example

Mammal can't access hunt and kill.



Base & Derived Classes

- A class can be derived from more than one classes, which means it can inherit data and functions from multiple base classes.
- To define a derived class, we use a **class derivation list** to specify the base class(es).
- A class derivation list names one or more base classes .
- Compiler : accessing base class functions when no such function available in derived class.

class derived-class: access-specifier base-class

Example 1:

```
#include <iostream>
using namespace std;
// Base class
class Employee {
  protected:
    int age;
    string name;
    float salary;
public:
    void setAge(int w)  {
      age = w;   }
    void setName(string h)  {
      name = h; }
     void setSalary(float f)  {
      salary = f;  }
};
// Derived class
class VIPEmployee: public Employee {
  public:
    float getBonus() {
        return (salary*1.5); }
};
int main(){
  VIPEmployee emp;
  emp.setAge(50);
  emp.setName("Ravi");
  emp.setSalary(40000.0);
cout << "Bonus: " << emp.getBonus() ;
  return 0;
}
```

- A derived class can access all the non-private members of its base class.
- Therefore base-class members that should not be accessible to the member functions of derived classes should be declared private in the base class.

Access	public	protected	private
Same class	yes	yes	yes
Derived classes	yes	yes	no
Outside classes	yes	no	no

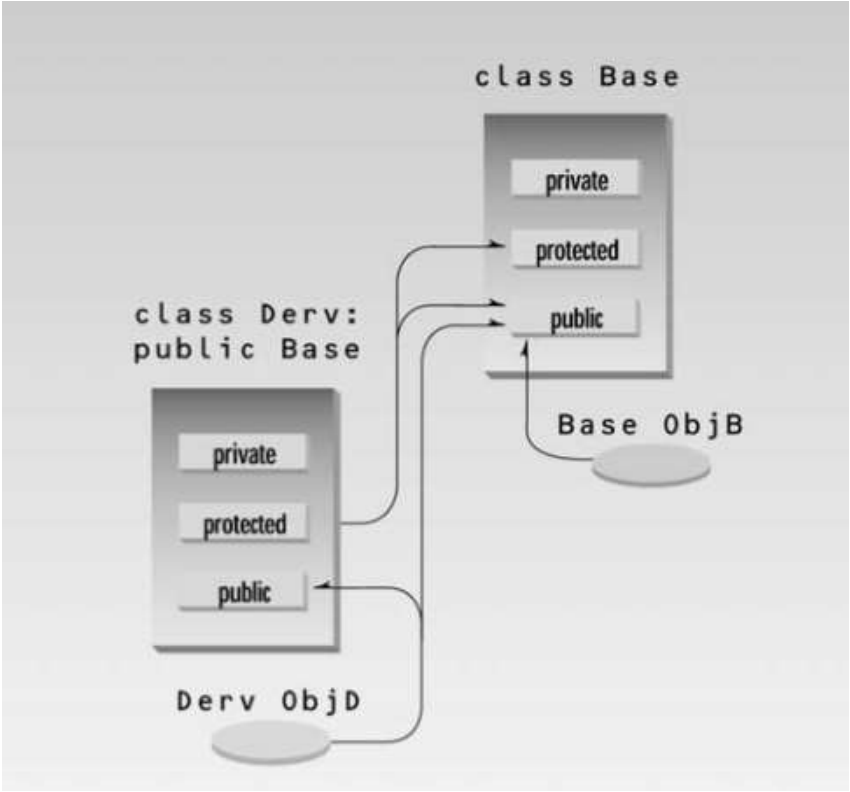
Example 2: Create a class called countdown to overload decrement operator and inherit features from Counter class

```
#include <iostream>
using namespace std;

class Counter
   {
   protected:
      unsigned int count;
   public:
      Counter() : count(0)
         {  }
      Counter(int c) : count(c)
         {  }
      unsigned int get_count() const
         { return count; }
      Counter operator ++ ()
         { return Counter(++count); }
   };

class CountDn : public Counter
   {
   public:
      Counter operator -- ()
         { return Counter(--count); }
   };
```

```
int main()
   {
   CountDn c1;

   cout << "\nc1=" << c1.get_count();

   ++c1; ++c1; ++c1;
   cout << "\nc1=" << c1.get_count();

   --c1; --c1;
   cout << "\nc1=" << c1.get_count();
   cout << endl;
   return 0;
   }
```

Access Specifiers with Inheritance

- Member functions in the derived class can access members of the base class if the members are public, or if they are protected. They can't access private members.
- A protected member can be accessed by member functions in its own class or and in any class derived from its own class. Not the other functions outside the class.



Derived Class Constructors

- Under certain circumstances—if you don't specify a constructor, the derived class will use an appropriate constructor from the base class.
- What happens if we want to initialize a Derived class object to a value??? Can't use Base class constructors.

Can add additional statements. Before execute constructor initialize values.

```
class Counter
   {
   protected:
      unsigned int count;
   public:
      Counter() : count()
         {  }
      Counter(int c) : count(c)
         {  }
      unsigned int get_count() const
         { return count; }
      Counter operator ++ ()
         { return Counter(++count); }
   };
```

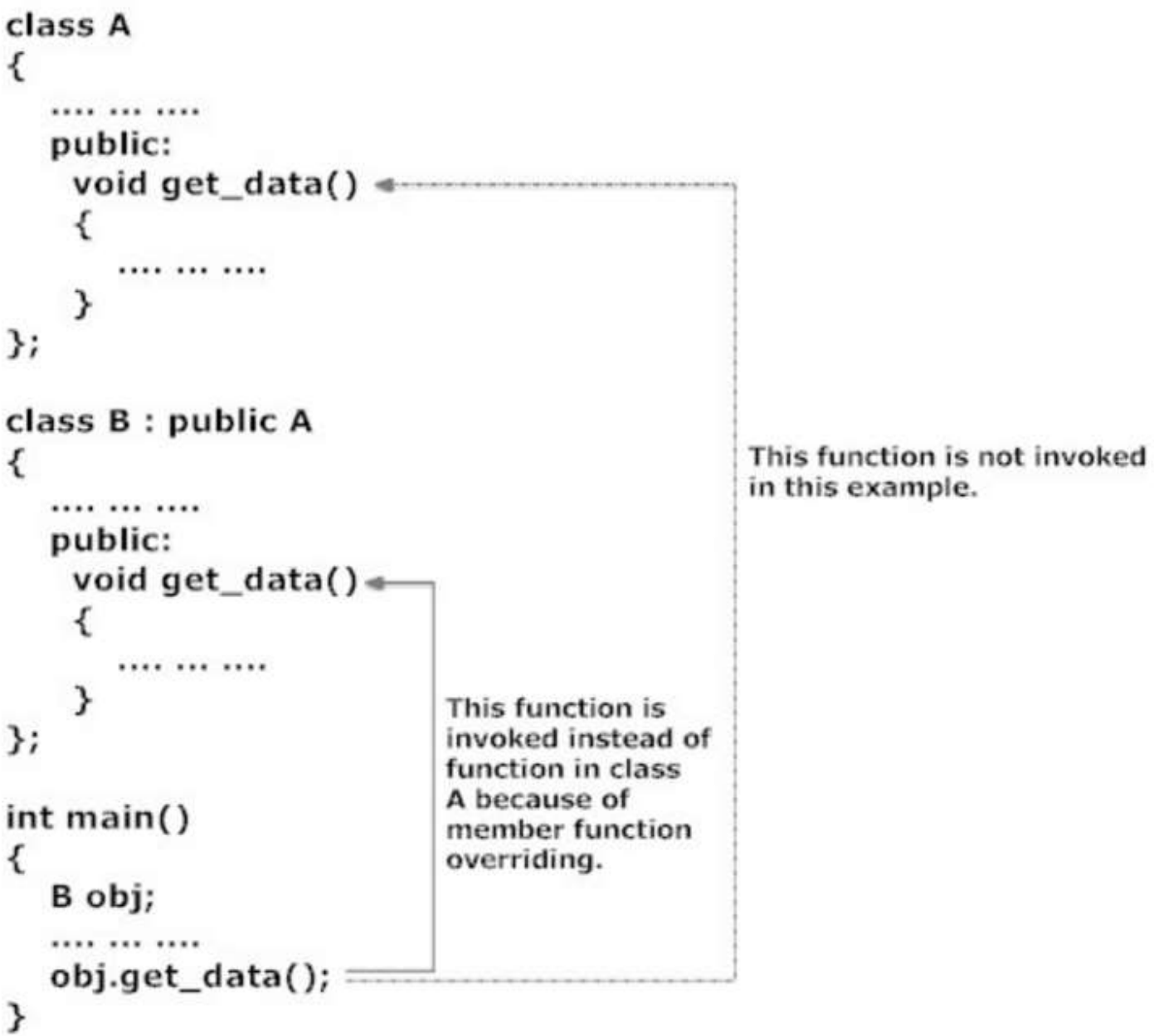
```
class CountDn : public Counter
   {
   public:
      CountDn() : Counter()
         { }
      CountDn(int c) : Counter(c)
         { }
      CountDn operator -- ()
         { return CountDn(--count); }
   };
```

In Main -:

```
CountDn c1;
CountDn c2(100);
```

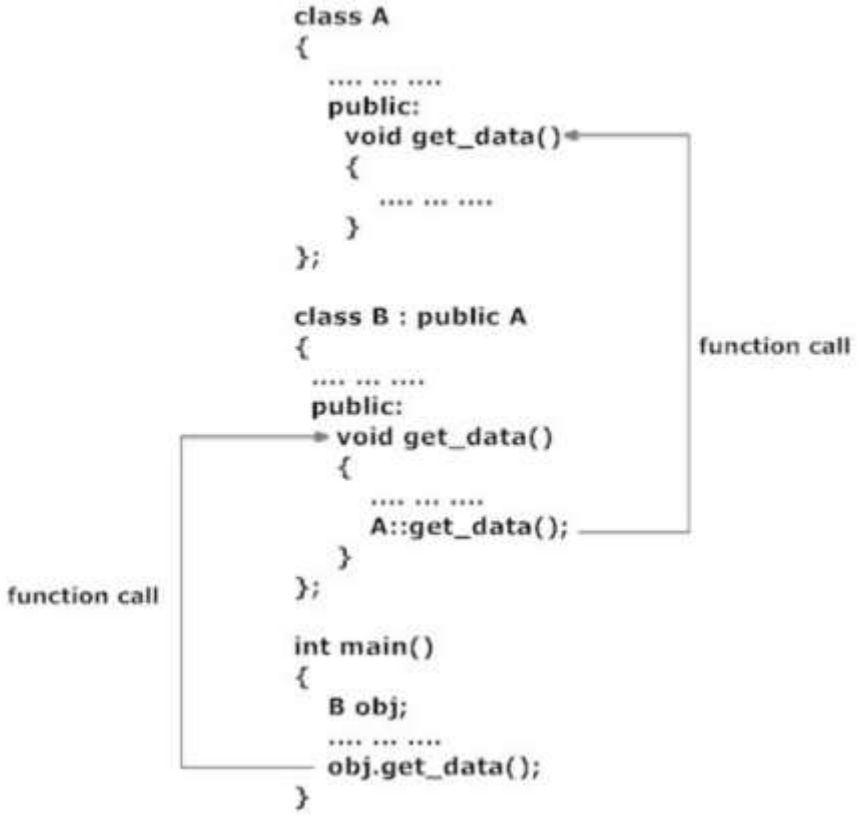
Overriding Member Functions

- Base class and derived class have member functions with same name and arguments(Same signature).
- If you create an object of derived class and write code to access that member function then, the member function in derived class is only invoked, i.e., the member function of derived class overrides the member function of base class.
- It is called function overriding.


class A
{
....
public:
void get_data()
{
....
}
};
class B : public A
{
....
public:
void get_data()
{
....
}
};
int main()
{
B obj;
....
obj.get_data();
}
This function is not invoked in this example.
This function is invoked instead of function in class A because of member function overriding.

Accessing the Overridden Function in Base Class

- Scope resolution operator :: id used to access the overridden function of base class from derived class.



Class Hierarchies

Q1. Implement a class named Employee with 2 data members , String type name and int type emp_no. Two member functions to get data and to set data. There are 3 types of Employees AcademicStaff , NonAcademicStaff and Laborers .

AcademicStaff: additionaly have a publicationNo which is type int.

NonAcademicStaff : additionaly have OT hours which is type float.

Laborers : does not have any additional information.

Abstract Class →

```
#include<iostream>
using namespace std;

class Employee{

    private:
        int emp_no;
        string name;
    public:
        void setdata(){
            cout<<"\nEnter the employee name: ";
            cin>>name;
            cout<<"Enter Employee No: ";
            cin>>emp_no;
        }

        void showdata(){
            cout<<"\n*********************************";
            cout<<"\nEmp Name:"<<name;
            cout<<"\nEmp No:"<<emp_no;

        }

        void showline(){
            cout<<"\n*********************************";
        }

};
```

```
class AcademicStaff : public Employee{

    private:
        int publications;
     public:
        void setdata(){
            Employee::setdata();
            cout<<"Enter No of Publications:";
            cin>>publications;
        }

        void showdata(){
            Employee::showdata();
            cout<<"\nPublication Count is:"<<publications;
        }


};

class NAcademicStaff : public Employee{

    private:
        float ot;
     public:
        void setdata(){
            Employee::setdata();
            cout<<"OT hours:";
            cin>>ot;
        }

        void showdata(){
            Employee::showdata();
            cout<<"\nOT hours:"<<ot;
        }


};
```

```
class Labourer : public Employee{};

int main(){

    Employee emp1;
    emp1.setdata();
    emp1.showdata();
    emp1.showline();

    AcademicStaff s1;
    s1.setdata();
    s1.showdata();
    s1.showline();

    NAcademicStaff s2;
    s2.setdata();
    s2.showdata();
    s2.showline();

    Labourer l1;
    l1.setdata();
    l1.showdata();
    l1.showline();

}
```

Type of Inheritance

- When deriving a class from a base class, the base class may be inherited through
  - **Public**
  - **Protected** or
  - **Private** inheritance.

  The type of inheritance is specified by the access-specifier.
- Hardly use protected or private inheritance, but public inheritance is commonly used.
- If you don't supply any access specifier when creating a class, private is assumed.

Public Inheritance:

- When deriving a class from a **public** base class,**public** members of the base class become **public** members of the derived class and **protected** members of the base class become **protected** members of the derived class.
- A base class's **private** members are never accessible directly from a derived class, but can be accessed through calls to the **public** and **protected** members of the base class.

Protected Inheritance:

- When deriving from a **protected** base class,**public** and **protected** members of the base class become **protected** members of the derived class.

Private Inheritance:

- When deriving from a **private** base class, **public** and **protected** members of the base class become **private** members of the derived class.

Example

```
#include <iostream>
using namespace std;

class A
   {
   private:
      int privdataA;
   protected:
      int protdataA;
   public:
      int pubdataA;
   };

 class B : public A
    {
    public:
       void funct()
          {
          int a;
          a = privdataA;    1
          a = protdataA;    2
          a = pubdataA;     3
          }
    };
```

```
class C : private A
   {
   public:
      void funct()
         {
         int a;
         a = privdataA;    4
         a = protdataA;    5
         a = pubdataA;     6
         }
   };
```

```
int main()
   {
   int a;


    B objB;
    a = objB.privdataA;    7
    a = objB.protdataA;    8
    a = objB.pubdataA;     9

    C objC;
    a = objC.privdataA;    10
    a = objC.protdataA;    11
    a = objC.pubdataA;     12
    return 0;
    }
```

What should be the results at each point from 1-12 ???

Levels of Inheritance

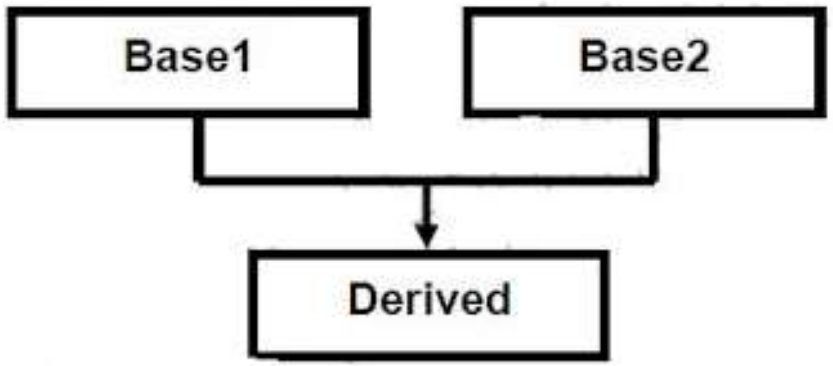
- Classes can be derived from classes that are themselves derived, this process can be extended to an arbitrary number of levels.

```
class A
   { };
class B : public A
   { };
class C : public B
   { };
```

Q2. Add a Registrar class which is a type of Non Academic Staff. Special features are they have a grade of type int and a unit which is type string.

Multiple Inheritance

- C++ class can inherit members from more than one class.



class derived-class: access baseA, access baseB...

Example 1:

```
#include <iostream>
using namespace std;
// Base class 1
class Person {
  protected:
    int age;
    string name;
  public:
    void setAge(int w)  {
      age = w;   }
    void setName(string h)  {
      name = h; }
};
```

```
// Base class 2
class Employee {
  protected:
        float salary;
public:
void setSalary(float f)  {
      salary = f;  }
};
```

```
// Derived class
class VIPEmployee: public Person, public
Employee {
  public:
    float getBonus() {
        return (salary*1.5); }
};
int main(){
  VIPEmployee emp;
  emp.setAge(50);
  emp.setName("Ravi");
  emp.setSalary(40000.0);
cout << "Bonus: " << emp.getBonus() ;
  return 0;
}
```

Example 2:

```
#include <iostream>
using namespace std;
// Base class Shape
class Shape {
  public:
    void setWidth(int w)  {
      width = w;
    }
    void setHeight(int h) {
      height = h;
    }
  protected:
    int width;
    int height;
};
```

```
// Base class PaintCost
class PaintCost {
  public:
    int getCost(int area) {
      return area * 70;
    }
};

// Derived class
class Rectangle: public Shape, public PaintCost {
  public:
    int getArea()
    {
      return (width * height);
    }
};
```

```
int main(){
   Rectangle Rect;
   int area;
   Rect.setWidth(5);
   Rect.setHeight(7);
   area = Rect.getArea();

   // Print the area of the object.
   cout << "Total area: " << Rect.getArea() << endl;

   // Print the total cost of painting
   cout << "Total paint cost: $" << Rect.getCost(area) << endl;

   return 0;
}
```

Ambiguity in Multiple Inheritance

1. Two base classes have functions with the same name, while a class derived from both base classes has no function with this name.

```
#include <iostream>
using namespace std;

class A
   {
   public:
      void show()  { cout << "Class A\n"; }
   };
class B
   {
   public:
      void show()  { cout << "Class B\n"; }
   };
class C : public A, public B
   {
   };
```

```
int main()
   {
   C objC;
// objC.show();        //Error
   objC.A::show();     //Ok
   objC.B::show();     //Ok
   return 0;
   }
```

2. If you derive a class from two classes that are each derived from the same class.

```
class A
   {
   public:
      void func();
   };
class B : public A
   {  };
class C : public A
   {  };
class D : public B, public C
   {  };
```

```
int main()
   {
   D objD;
   objD.func();  //ambiguous: won't compile
   return 0;
   }
```

Try it…

“MyBooks” is a publishing company that markets both book and audiocassette versions of its works. Create a class publication that stores the title (a string) and price (type float) of a publication. From this class derive two classes: book, which adds a page count (type int), and tape, which adds a playing time in minutes (type float). Each of these three classes should have a getdata() function to get its data from the user at the keyboard, and a putdata() function to display its data.

Add a base class sales that holds an array of three floats so that it can record the dollar sales of a particular publication for the last three months. Include a getdata() function to get three sales amounts from the user, and a putdata() function to display the sales figures. Alter the book and tape classes so they are derived from both publication and sales. An object of class book or tape should input and output sales data along with its other data. Write a main() function to create a book object and a tape object and exercise their input/output capabilities.

Friendship

- In principle, private and protected members of a class cannot be accessed from outside the class in which they are declared. However, this rule does not apply to "friends".
- Friends are functions or classes declared with the **friend** keyword.

Friend functions

- A non-member function can access the private and protected members of a class if it is declared a friend of that class.
- That is done by including a declaration of this external function within the class, and preceding it with the keyword friend:

```
#include <iostream>
using namespace std;

class Rectangle {

    int width, height;

  public:
    Rectangle() {}
    Rectangle (int x, int y) : width(x), height(y) {}
    int area() {return width * height;}
    friend Rectangle duplicate (const Rectangle&);
};

Rectangle duplicate (const Rectangle& param)
{
  Rectangle res;
  res.width = param.width*2;
  res.height = param.height*2;
  return res;
}

int main () {
  Rectangle foo;
  Rectangle bar (2,3);
  foo = duplicate (bar);
  cout << foo.area() << '\n';
  return 0;
}
```

- Notice though that neither in the declaration of duplicate nor in its later use in main, function duplicate is considered a member of class Rectangle.
- It simply has access to its private and protected members without being a member.
- Typical use cases of friend functions are operations that are conducted between two different classes accessing private or protected members of both.

Friend classes

- A friend class is a class whose members have access to the private or protected members of another class.
- If we make the whole class a friend class, al the members can access to private and protected members.

Friend classes

- A friend class is a class whose members have access to the private or protected members of another class.

```
#include <iostream>
using namespace std;

class Square;

class Rectangle {
    int width, height;
  public:
    int area ()
      {return (width * height);}
    void convert (Square a);
};

class Square {
  friend class Rectangle;
  private:
    int side;
  public:
    Square (int a) : side(a) {}
};

void Rectangle::convert (Square a) {
  width = a.side;
  height = a.side;
}

int main () {
  Rectangle rect;
  Square sqr (4);
  rect.convert(sqr);
  cout << rect.area();
  return 0;
}
```

- In this example, class Rectangle is a friend of class Square allowing Rectangle's member functions to access private and protected members of Square. (Rectangle accesses the member variable Square::side, which describes the side of the square.)
- At the beginning of the program, there is an empty declaration of class Square. This is necessary because class Rectangle uses Square (as a parameter in member convert), and Square uses Rectangle (declaring it a friend).
- Friendships are never corresponded unless specified.
- Another property of friendships is that they are not transitive: The friend of a friend is not considered a friend unless explicitly specified.

Thank You !